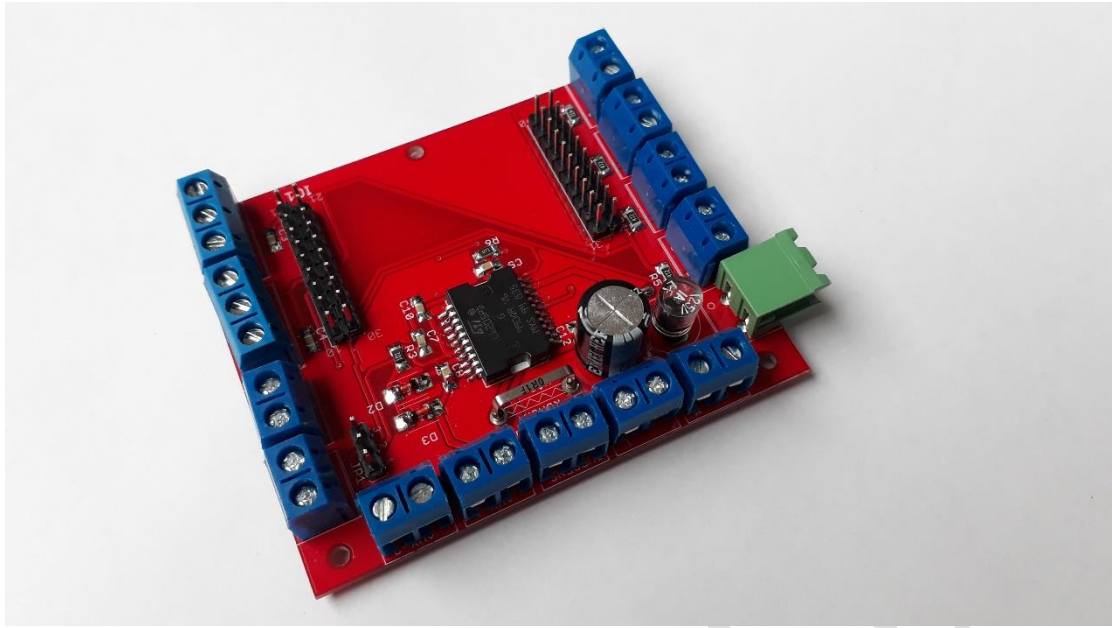




Tarjeta de control motor

Documento de presentación

Presentación

La tarjeta MC2 es una tarjeta de control de motores DC para la TIVA EK-TM4C123GXL LaunchPad de Texas Instruments, que tiene las características principales siguientes:

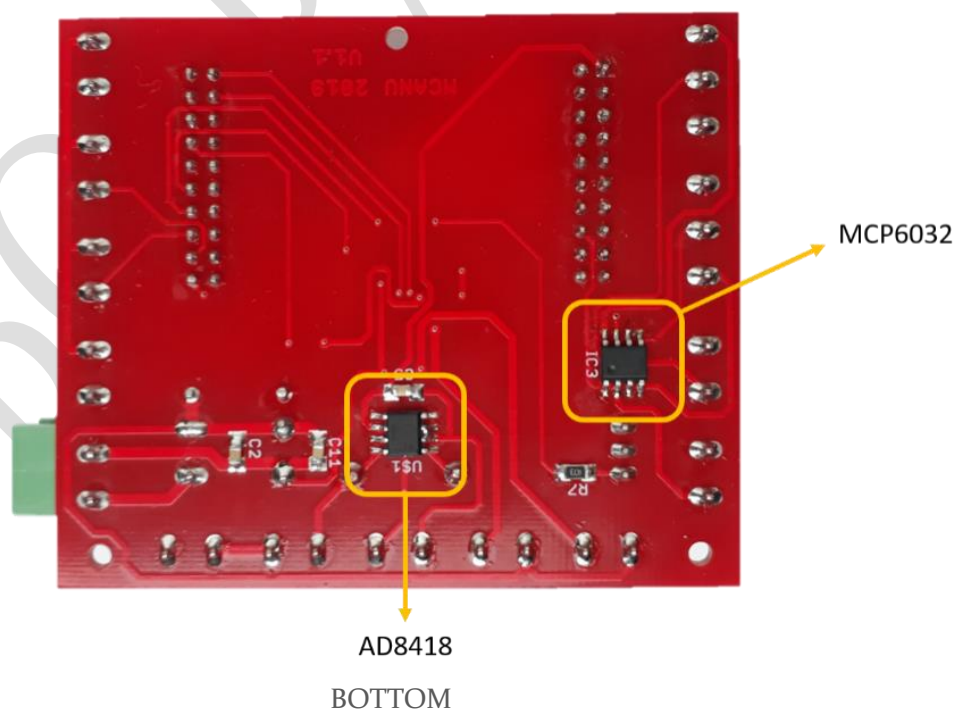
- Salida 1 motor DC + 2 periféricos o 2 motores DC: 8-25 V / 2.8 A (CC)
- 2 entradas análogas
- 4 entradas digitales
- Salida de medición de corriente motor para la salida de potencia principal.
- Protección contra cortos circuitos (fusibles) y sobre calentamiento (L6205)

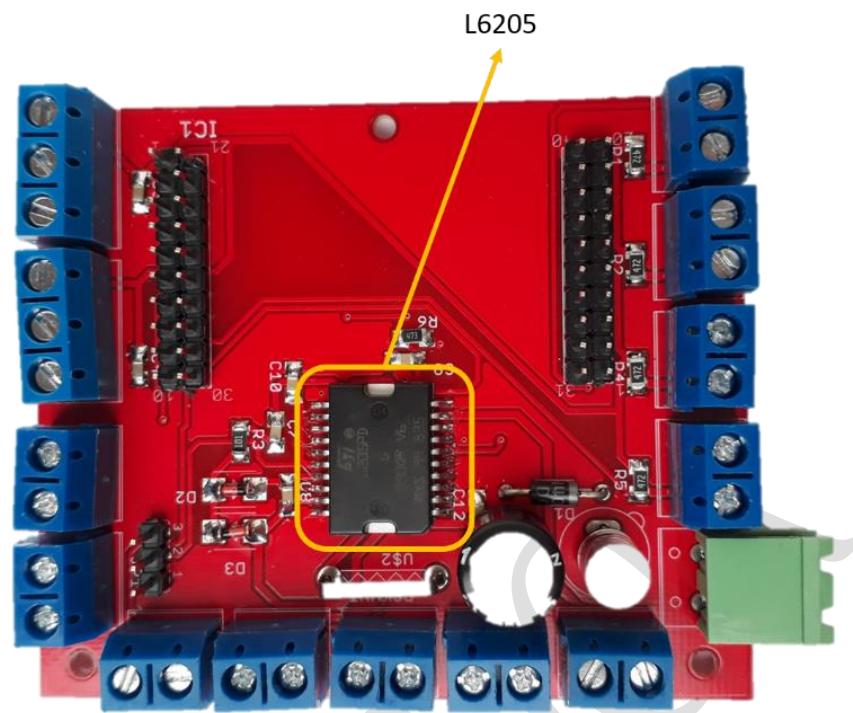
La tarjeta incluye un puente H moderno en tecnología MOSFET L6205 capaz de soportar una frecuencia de PWM hasta 100KHz. Tiene una conexión externa para poder ajustar la inductancia serie del motor en función de la frecuencia de la PWM y de las características del motor. La tarjeta permite la conexión de un sensor analógico de posición o velocidad (0-3.3V) para realizar una realimentación de salida, y también una medición del corriente motor (vía un AOP específico AD8418) para poder implementar una realimentación de estado. Se puede conectar entradas digitales de fin de carrera, por ejemplo.

PCB y ubicación de componentes

El PCB esta de doble capa, pistas de 17 micras.

Componentes principales:

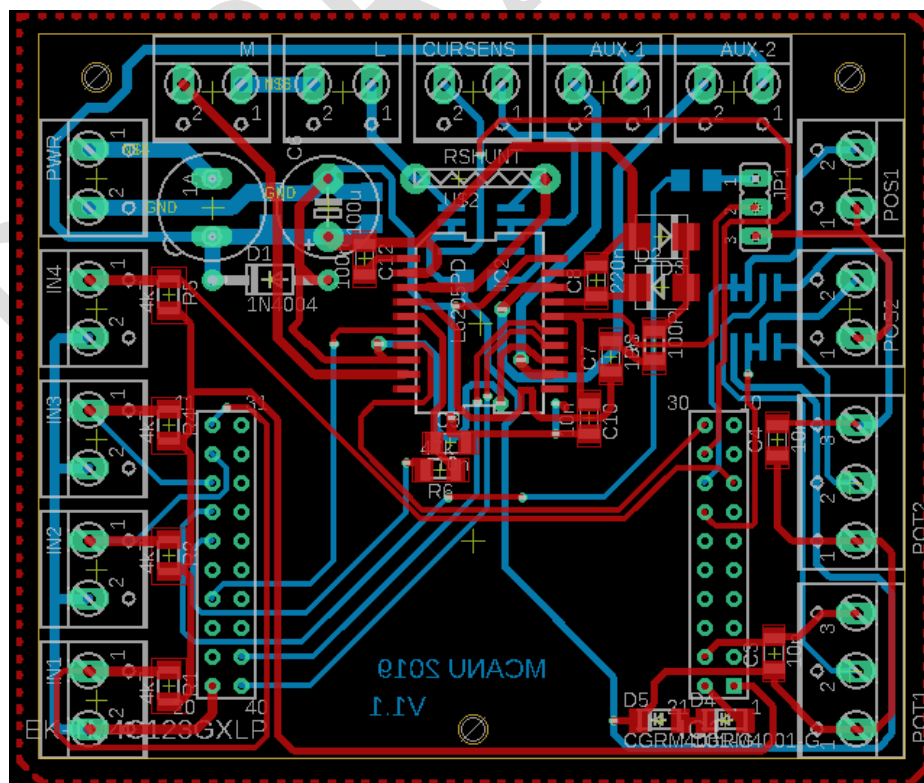




TOP

Diseño del PCB

5



5

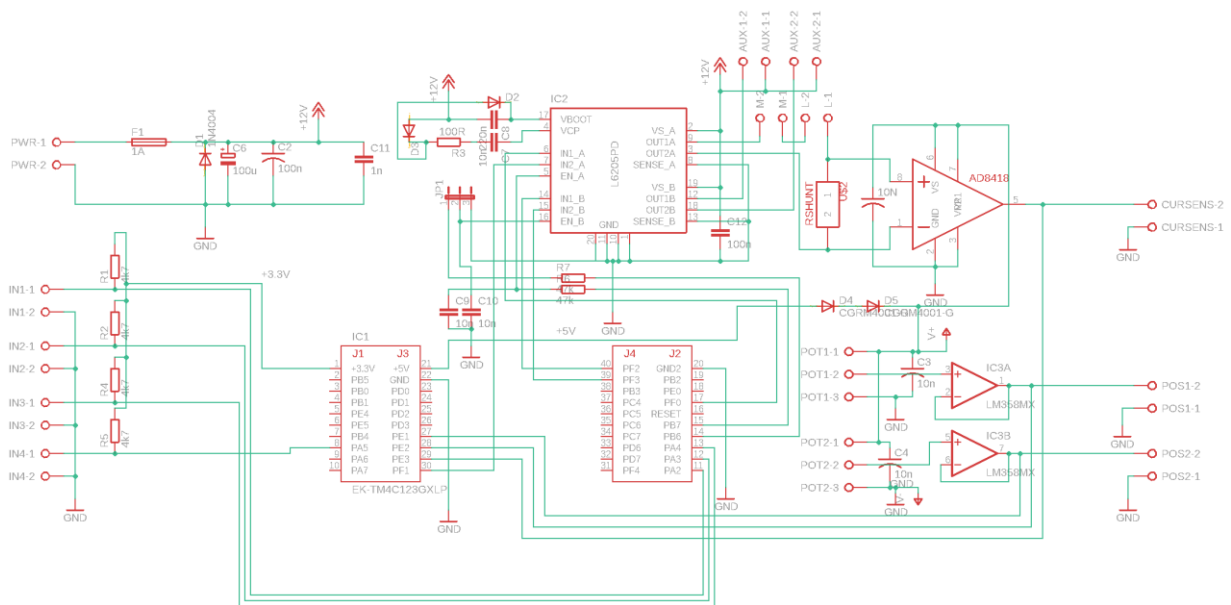
Entrées analogiques :

POT₁ : PE₂

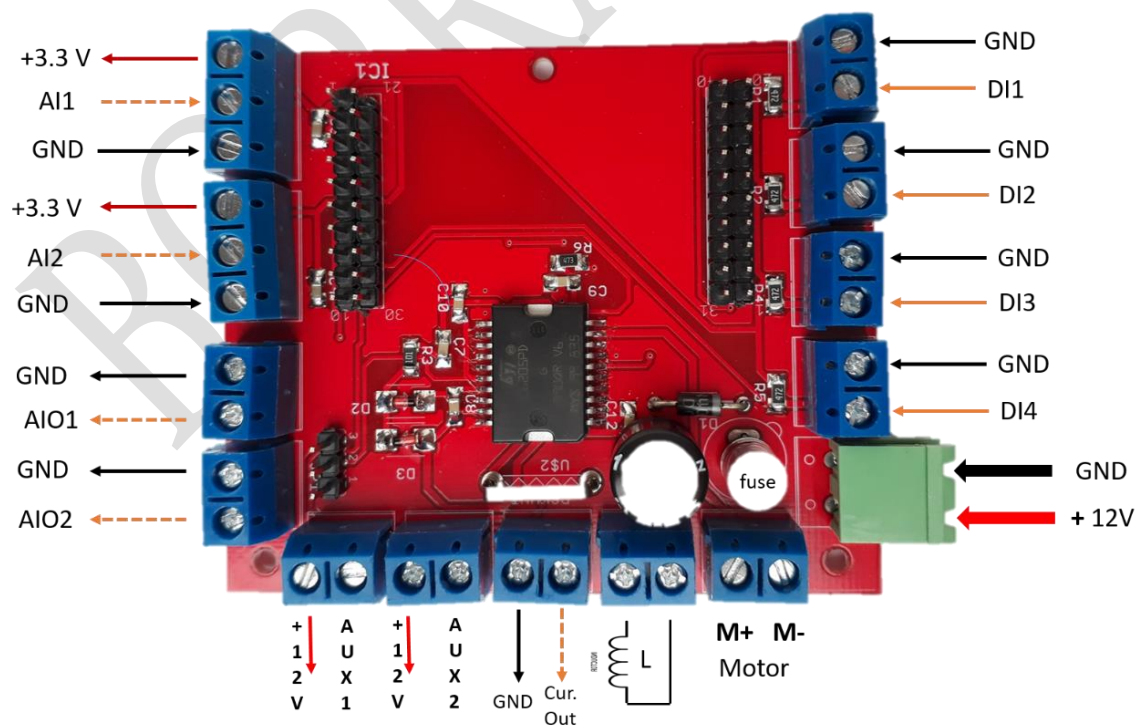
POT₂ : PE₁

BORRADOR

Diagrama



Conexiones



NOTA: AIO1, AIO2 y Cur. Out son salidas conectadas a la tarjeta TIVA internamente.

Aplicación en laboratorios de control

Control Motor DC, por realimentación de salida o de estado:

- continuo a través de Matlab/Simulink (comunicación serie)
- implementación discreta en C

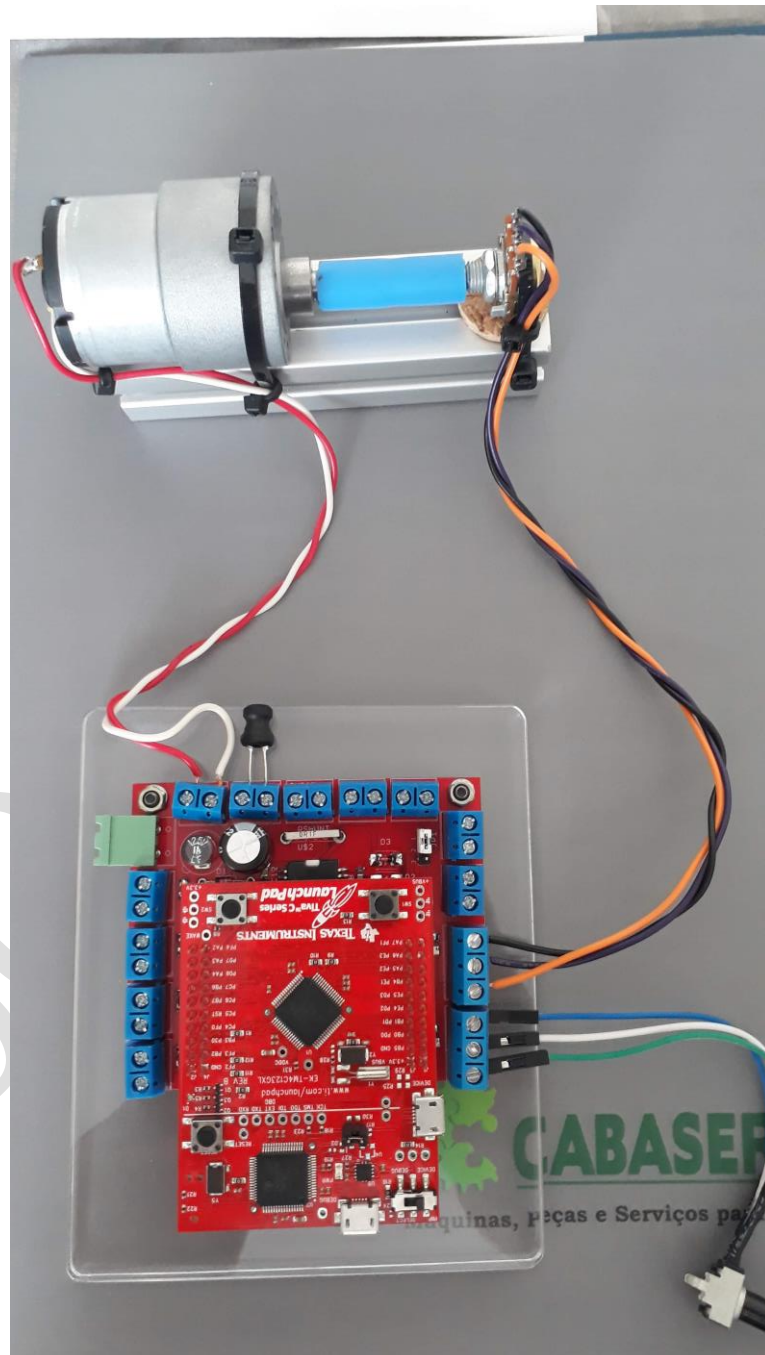


Ilustración 1: Sistema completo de regulación de posición de motor DC



Costo unitario:

PCB: 15800

Puente H: 21000

AD8418: 10000

MCP6032: 3200

Otros componentes: 10000

Total: 60.000,00

TIVA TM4C123: 48000

Total: 108.000,00

Annexe 1 : TIVA CODE

```
/*
    Motor Control for TIVA C
    MICHAEL CANU
*/

#include <stdbool.h>
#include <stdio.h>
#include <math.h>
#include "inc/hw_ints.h"
#include "inc/hw_memmap.h"
#include "inc/hw_types.h"
#include "driverlib/gpio.h"
#include "driverlib/sysctl.h"
#include "driverlib/systick.h"
#include "driverlib/pin_map.h"
#include "inc/hw_gpio.h"
#include "driverlib/pwm.h"
#include "driverlib/qei.h"
#include "driverlib/timer.h"
#include "driverlib/interrupt.h"
#include "driverlib/eeeprom.h"

// DEFINICION ENTRADAS/SALIDAS
#define SensorPos1 PE_1 //entrada sensor de posición 1
#define SensorPos2 PE_2 //entrada sensor de posición 1
#define SensorCur PE_3 //entrada sensor de corriente (AD)

// #define ENABLE_M1 PB_7 //salida validación puente 1 (motor)
// #define ENABLE_M2 PB_6 //salida validación puente 2 (aux)

#define SW1 PF_4 //entrada switch validación potencia
#define DI1 PA_2
#define DI2 PA_3
#define DI3 PA_4
#define DI4 PA_5

int ENABLE_M1 = PB_7; //PB_7 salida validación puente 1 (motor)
int ENABLE_M2 = PB_6; //PB_6 salida validación puente 2 (aux)

int RSTpin = 10;
int POS, REF, ER, CUR;

int initPOS = 25; //initial position
int minPOS = 0; //initial min position
int maxPOS = 3125; //initial max position
const int distappr = 500; //low speed distance
const int amplitud = 250; //degree amplitud of rotation
const int MinSimTime = 100; //Minimum simulation time = 4*MinSimTime
const float lowspeedminPOS = 65.0; //0-40-45--50--55-60-100
const float speedminPOS = 70.0;
const float lowspeedmaxPOS = 35.0;
const float speedmaxPOS = 30.0;
```

```

float coeff, coeff_curr;

int error_var = 30; //steady state position variation in bits (->4049): 0.0068°/bits
int OpenLoopSampling = 5; //sampling for open loop (2-10)

int PWM_value = 127; //initial value = medium speed

int SetPoint, L_ERR, ERR, Ted = 0;

float COM, U, INT, Te, Kp, Ki, Kd, Ad, Bd, Cd = 0.0;

float DER = 0.0;

int TP1, TP2 = 0;

int y[420]; //position
int x[420]; //time
int z[420]; //current

unsigned long t_init;

int t;

int StepInitTime = 50; //step time in ms

int Timetable[2];

int i,j = 0;

int PERIOD; //PERIOD PWM
int PULSEWIDTH; //PULSE HIGH LEVEL TIME
float DutyCycle; //Between 0 and 100

const int FREQ = 11000; //frecuencia en Hz
bool Mot_Dir = false; //motor direction memory
// pins for the LEDs:
const bool ERROR = 0;

int out_rec = 0; //define if output derivative or current is require
bool tresD = false; //send 3D table or 2D table
bool AWU = false; //anti-windup on/off

void PWMsetDC(float Duty){
    PULSEWIDTH = int(PERIOD * Duty / 100);
    PWMPulseWidthSet(PWM1_BASE, PWM_OUT_4, PULSEWIDTH);
}

int sign(float v) {
    return (v < 0) ? -1 : ((v > 0) ? 1 : 0);
}

// send 2 dimensions table in JSON format: x = time, y = position or velocity
void print2DtableJSON(int *tableauX, int *tableauY, int *tableauZ, int tailleTableau)
{
    /** SEND JSON DATA FOR POSITION AND TIME **
    Serial.print('[');
    //First values
    Serial.print('{');
    Serial.print(F("\x\"x\":"));
    Serial.print(tableauX[0]); //Time
    Serial.print(',');
    Serial.print(F("\x\"y\":"));
    if (out_rec == 2) { //current
        Serial.print(tableauZ[0]*coeff_curr); //motor current
    } else { //speed or position
        Serial.print(tableauY[0]*coeff); //position
    }
    Serial.print('}');
    Serial.print(',');

```

```

//for values after 0
for (j = 1 ; j < tailleTableau-1 ; j++)
{
    Serial.print('{}');
    Serial.print(F("\'x\':"));
    Serial.print(tableauX[j]);
    Serial.print(',');
    Serial.print(F("\'y\':"));
    if (out_rec == 1) { //if the user want the derivate of the output
        Serial.print((tableauY[j+1]-tableauY[j-1])*coeff*1000/(tableauX[j+1]-tableauX[j-1]));
    } else if (out_rec == 2) { //if the user want the motor current
        Serial.print(tableauZ[j]*coeff_curr);
    } else {
        Serial.print(tableauY[j]*coeff); //if the user want the position
    }
    Serial.print('{}');
    Serial.print(',');
}

//for the ultimate value, only the position is send to the server
Serial.print('{}');
Serial.print(F("\'x\':"));
Serial.print(tableauX[tailleTableau-1]); //no matter the time
Serial.print(',');
Serial.print(F("\'y\':"));
Serial.print(tableauY[tailleTableau-1]*coeff); //ultimate position
Serial.print('{}');
Serial.println('{}');
}

// send 3 dimensions table in JSON format: x = time, y = position , z = command
void print3DtableJSON(int *tableauX, int *tableauY, int *tableauZ, int tailleTableau)
{
    /** SEND JSON DATA FOR POSITION AND TIME **/
    Serial.print('{}');
    //First values
    Serial.print('{}');
    Serial.print(F("\'x\':"));
    Serial.print(tableauX[0]); //Time
    Serial.print(',');
    Serial.print(F("\'y\':"));
    Serial.print(tableauY[0]*coeff); //position
    Serial.print(',');
    Serial.print(F("\'z\':"));
    Serial.print(-(tableauZ[0]-50)*0.24); //command
    Serial.print('{}');
    Serial.print(',');
    //for values after 0
    for (j = 1 ; j < tailleTableau-1 ; j++)
    {
        Serial.print('{}');
        Serial.print(F("\'x\':")); //time
        Serial.print(tableauX[j]);
        Serial.print(',');
        Serial.print(F("\'y\':"));
    }
}

```

```

        Serial.print(tableauY[j]*coeff); //the position
        Serial.print(',');
        Serial.print(F("\nz\n:"));
        Serial.print(-(tableauZ[j]-50)*0.24); //command
        Serial.print('');
        Serial.print(',');
    }

    //for the ultimate value, only the position is send to the server
    Serial.print('{');
    Serial.print(F("\nx\n:"));
    Serial.print(tableauX[tailleTableau-1]); //no matter the time
    Serial.print(',');
    Serial.print(F("\ny\n:"));
    Serial.print(tableauY[tailleTableau-1]*coeff); //ultimate position
    Serial.print(',');
    Serial.print(F("\nz\n:"));
    Serial.print(-(tableauZ[tailleTableau-1]-50)*0.24); //ultimate position
    Serial.print('');
    Serial.println(']');
}

void setup() {
    //digitalWrite(RSTpin, HIGH);
    delay(200);
    // initialize serial:
    Serial.begin(115200);

    // CONFIGURACION ENTRADAS y SALIDAS
    pinMode(ENABLE_M1, OUTPUT);
    pinMode(ENABLE_M2, OUTPUT);

    pinMode(DI1, INPUT);
    pinMode(DI2, INPUT);
    pinMode(DI3, INPUT);
    pinMode(DI4, INPUT);

    pinMode(SW1, INPUT_PULLUP);

    pinMode(SensorPos1, INPUT);
    pinMode(SensorPos2, INPUT);
    pinMode(SensorCur, INPUT);

    // ENABLE MOTOR
    digitalWrite(ENABLE_M1, LOW);
    digitalWrite(ENABLE_M2, LOW);

    // ***** PWM INICIALIZACION *****
    // *** frecuencia = FREQ, ciclo util = DutyCycle ***

    PERIOD = 80000/(FREQ/1000); // STEPS = PERIOD / PREC
    DutyCycle = 50.0; // duty cycle en percentage 0-100 float
    PULSEWIDTH = int(PERIOD * DutyCycle / 100);

```

```

SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOF);
SysCtlPeripheralEnable(SYSCTL_PERIPH_PWM1);
SysCtlPWMClockSet(SYSCTL_PWMDIV_1);
HWREG(GPIO_PORTF_BASE + GPIO_O_LOCK) = GPIO_LOCK_KEY;
HWREG(GPIO_PORTF_BASE + GPIO_O_CR) |= 0x01;
GPIOPinConfigure(GPIO_PF0_M1PWM4);
GPIOPinConfigure(GPIO_PF1_M1PWM5);
GPIOPinConfigure(GPIO_PF2_M1PWM6);
GPIOPinConfigure(GPIO_PF3_M1PWM7);
GPIOPinTypePWM(GPIO_PORTF_BASE, GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_2 | GPIO_PIN_3);
PWMGenConfigure(PWM1_BASE, PWM_GEN_2, PWM_GEN_MODE_DOWN | PWM_GEN_MODE_NO_SYNC);
//PWMGenConfigure(PWM1_BASE, PWM_GEN_3, PWM_GEN_MODE_DOWN | PWM_GEN_MODE_NO_SYNC);

PWMDeadBandEnable(PWM1_BASE, PWM_GEN_2, 10, 10); // Coupling the two outputs PWM_OUT_5 = /PWM_OUT_4

PWMGenPeriodSet(PWM1_BASE, PWM_GEN_2, PERIOD);

PWMPulseWidthSet(PWM1_BASE, PWM_OUT_4, PULSEWIDTH);
//PWMPulseWidthSet(PWM1_BASE, PWM_OUT_5, PULSEWIDTH);

PWMGenEnable(PWM1_BASE, PWM_GEN_2);
PWMOutputState(PWM1_BASE, (PWM_OUT_4_BIT | PWM_OUT_5_BIT | PWM_OUT_6_BIT | PWM_OUT_7_BIT), true);

//SysCtlClockSet(SYSCTL_SYSDIV_4|SYSCTL_USE_PLL|SYSCTL_XTAL_16MHZ|SYSCTL_OSC_MAIN);

/** automatic coefficient setup **
// increasing angle
POS = analogRead(SensorPos1); //get initial position
PWMsetDC(speedmaxPOS); //Set medium speed val < 50 => go far away from 0
digitalWrite(ENABLE_M1, HIGH); //enable motor 1
t_init = millis(); //get current time
while (POS < maxPOS-1) { //move until initial position is reached
    delay(10);
    POS = analogRead(SensorPos1); //get position
    if (POS > maxPOS-distappr) { //near max position
        PWMsetDC(lowspeedmaxPOS); //set low speed rotation
    }
    if ((millis()-t_init) > 3000) {break;}; //whatchdog 3s
    //Serial.println(POS); //to be removed
}
digitalWrite(ENABLE_M1, LOW); //disable motor 1
delay(200); //wait for position stabilization
maxPOS = analogRead(SensorPos1);
Serial.println(maxPOS);
delay(500);

// decreasing angle
POS = analogRead(SensorPos1); //get initial position
PWMsetDC(speedminPOS); //Set medium speed val > 50 => return to 0
digitalWrite(ENABLE_M1, HIGH); //enable motor 1
t_init = millis(); //get current time

```

```

while (POS > minPOS+1) {          //move until initial position is reached
    delay(10);
    POS = analogRead(SensorPos1); //get position
    if (POS < minPOS + distappr) { //near 0
        PWMsetDC(lowspeedminPOS); //set low speed rotation
    }
    if ((millis()-t_init) > 3000) {break;}; //whatchdog 3s
    //Serial.println(POS); //to be removed
}

digitalWrite(ENABLE_M1,LOW); //disable motor 1
delay(100); //wait for position stabilization
minPOS = analogRead(SensorPos1);
Serial.println(minPOS);

coeff = amplitud/float(maxPOS-minPOS); //coeff °/bits
coeff_curr = 20*3.3/4096; // coeff V/Amp
initPOS = minPOS;
}

void loop() {
    // if there's any serial available, read it:
    while (Serial.available() > 0) {

        // look for the next valid integer in the incoming serial stream:
        int command = Serial.parseInt();
        // look for the newline. That's the end of your sentence:
        if (Serial.read() == '\n') {
            //Serial.println(command);
            switch (command){

                case 0: //***** RETURN TO 0 INITIAL POSITION *****
                    POS = analogRead(SensorPos1); //get initial position
                    PWMsetDC(speedminPOS); //Set medium speed val > 50 => return to 0
                    digitalWrite(ENABLE_M1,HIGH); //enable motor 1
                    t_init = millis(); //get current time
                    while (POS > minPOS+1) {          //move until initial position is reached
                        delay(1);
                        POS = analogRead(SensorPos1); //get position
                        if (POS < 1000) {
                            PWMsetDC(lowspeedminPOS); //set low speed rotation
                        }
                        if ((millis()-t_init) > 2000) {break;}; //whatchdog 3s
                        //Serial.println(POS); //to be removed
                    }
                    digitalWrite(ENABLE_M1,LOW); //disable motor 1
                    delay(100); //wait for position stabilization
                    POS = analogRead(SensorPos1);
                    Serial.println(POS*coeff); //send final position to server
                    break;

                case 1: //***** OPEN LOOP STEP RESPONSE *****
                    PWMsetDC(50-(PWM_value*0.196)); //set Pwm value < 50 => go far away 0
                    x[0] = 0; //intial time of step

```

```

t_init = millis(); //get current time
y[0] = analogRead(SensorPos1); //initial position value
z[0] = analogRead(SensorCur); //initial current value
delay(StepInitTime); //wait before step start
digitalWrite(ENABLE_M1,HIGH); //enable motor 1
for (i = 1; i<300; i++){ //200 points max x 2-10 ms
    x[i] = int(millis()-t_init);
    y[i] = analogRead(SensorPos1);
    z[i] = analogRead(SensorCur);
    delay(OpenLoopSampling*i); //to increase sample time
    if ((i > 20) && (abs(y[i] - y[i-1]) < error_var) && (abs(y[i-2] - y[i-3]) < error_var)){
        //Serial.println(i); //to be removed
        break; // if steady state or a physical limit is reach
    }
}
digitalWrite(ENABLE_M1,LOW); //disable motor 1
//send output or output derivative
print2DtableJSON(x,y,z,i); //send time, position and current if required
//Serial.println(OpenLoopSampling);
break;

case 2: //***** CLOSE LOOP STEP RESPONSE PID *****
    PWMsetDC(50.0); // set Pwm dutycycle value to 50% = no torque
    i = 1; //index init
    j = 1;
    Timetable[0] = 0; //ellapse time index init (last time index)
    Timetable[1] = 0; //ellapse time index init (current time index)
    //digitalWrite(DIR,LOW); //set direction to anticlockwise
    INT = 0; //error integral initialization
    L_ERR = 0; //last error initialization
    x[0] = 0; //initial time
    t_init = millis(); //init current time
    y[0] = analogRead(SensorPos1); //initial position
    z[0] = 0; //initial command
    //Serial.println(POS); //to be removed
    digitalWrite(ENABLE_M1,HIGH); //enable motor 1
    //Serial.println(i);
    delay(StepInitTime); //wait before step start
    while (i < 1500) { //time limit for control loop in ms => 1,5s
        Timetable[1] = int(millis()-t_init); //get ellapse time
        POS = analogRead(SensorPos1); //get current position
        CUR = analogRead(SensorCur); //get motor current
        ERR = SetPoint - POS; //Error evaluation
        Te = (float(Timetable[1] - Timetable[0])/1000); //get loop period
        INT = INT + Ki*Te*float(ERR); //Integral calculus of ERR in ms
        DER = Kd*float(ERR-L_ERR)/Te; //Derivative calculus of ERR in ms
        COM = (-coeff*(float(ERR)*Kp + INT + DER)*4.16)+50; //PID parallel calculus 100/(2*amplitud)
        if (COM > 99.9){ //PWM limits
            COM = 99.9;
            if (AWU){INT = 0.0;}; //anti-windup
        } else if (COM < 0.1){
            COM = 0.1;
            if (AWU){INT = 0.0;}; //anti-windup
        }
    }
}

```

```

    };

    PWMsetDC(COM);    //update Pwm value

    if ((i-1) % 4) == 0){    //1 record data each 4

        x[j] = Timetable[1];

        y[j] = POS;

        z[j] = int(COM);

        j++;

    }

    TP1 = y[j-1] - y[j-3];    //ultimate output position difference
    TP2 = y[j-3] - y[j-6];    //last output position difference

    if ((j > MinSimTime) && (abs(TP1) < error_var) && (abs(TP2) < error_var)){

        //Serial.println("STOP"); //to be removed

        break;    // STOP if steady state or a physical limit is reach

    }

    L_ERR = ERR;                //error update

    Timetable[0] = Timetable[1];    //Ellapse time update

    //delayMicroseconds(500);    //loop delay

    delay(1);                //loop delay

    i++;                //Next step

}

digitalWrite(ENABLE_M1,LOW); //disable motor 1

//Serial.println(j);

print3DtableJSON(x,y,z,j); //send time, position and 10 x command

break;

case 3:    //***** CLOSE LOOP STEP RESPONSE SS *****

    //Serial.println("Step close loop space state");

    //Serial.println(F("OK"));

    break;

case 4: //***** Set PWM value *****

    //Serial.println("Set PWM"); //to be removed

    while (Serial.available() == 0){}; //wait port free

    while (Serial.available() > 0) { //byte reception

        PWM_value = Serial.parseInt();    //value between 0 and 255

        if (Serial.read() == '\n') {

            //Serial.println(PWM_value);

            break;

        }

    }

    //no return

    break;

case 5: //***** ENABLE MOTOR ROT *****

    //Serial.println("Enable motor rotation"); //to be removed

    if (Mot_Dir){

        PWMsetDC(PWM_value*0.39+50);    //set Pwm value > 50 => go to 0 position

    } else {

        PWMsetDC(50 - PWM_value*0.39);    //set Pwm value > 50 => go to 0 position

    }

    digitalWrite(ENABLE_M1,HIGH); //enable motor 1

    //no return

    break;

```

```

case 6: //***** DISABLE MOTOR ROT *****
    //Serial.println("Disable motor rotation"); //to be removed
    PWMsetDC(50.0); //50% duty cycle = stop motor
    digitalWrite(ENABLE_M1,LOW); //disable motor 1
    delay(150);
    POS = analogRead(SensorPos1);
    Serial.println(POS*coeff); //send current position
    break;

case 7: //***** DIRECTION TO ANTICLOCKWISE *****
    //Serial.println("Rotation anticlockwise"); //to be removed
    Mot_Dir = true; //PWM > 50 => return to 0 position
    //no return
    break;

case 8: //***** DIRECTION TO CLOCKWISE *****
    //Serial.println("Rotation clockwise"); //to be removed
    Mot_Dir = false; //PWM < 50 => go far away 0 position
    //no return
    break;

case 9: //***** SET Kp VALUE *****
    //Serial.println("Set Kp"); //to be removed
    while (Serial.available() == 0){}; //wait port free
    while (Serial.available() > 0) { //byte reception
        Kp = Serial.parseFloat(); //Kp received
        if (Serial.read() == '\n') {
            if (Kp > 100){Kp = 100;};
            if (Kp < 0){Kp = 0;};
            break;
        }
    }
    //no return
    break;

case 10: //***** SET Ki VALUE *****
    //Serial.println("Set Ki"); //to be removed
    while (Serial.available() == 0){}; //wait port free
    while (Serial.available() > 0) { //byte reception
        Ki = Serial.parseFloat(); //Ki
        if (Serial.read() == '\n') {
            if (Ki > 100){Ki = 100;};
            if (Ki < 0){Ki = 0;};
            //Serial.println(Kp);
            break;
        }
    }
    //no return
    break;

case 11: //***** SET Kd VALUE *****
    //Serial.println("Set Kd"); //to be removed

```

```

while (Serial.available() == 0){}; //wait port free
while (Serial.available() > 0) { //byte reception
    Kd = Serial.parseFloat(); //Kd
    if (Serial.read() == '\n') {
        if (Kd > 100){Kd = 100;};
        if (Kd < 0){Kd = 0;};
        //Serial.println(Kp);
        break;
    }
}
//no return
break;

case 12: //***** GET MOTOR POSITION *****
    //Serial.println("Get Position"); //to be removed
    POS = analogRead(SensorPos1);
    //Serial.println(POS);
    Serial.println(POS*coeff);
    break;

case 13: //***** SET CONTROL SETPOINT *****
    //Serial.println("Set SetPoint"); //to be removed
    while (Serial.available() == 0){}; //wait port free
    while (Serial.available() > 0) { //byte reception
        SetPoint = int(Serial.parseFloat()/coeff);
        if (Serial.read() == '\n') {
            if (SetPoint < minPOS) {SetPoint = minPOS;};
            if (SetPoint > maxPOS) {SetPoint = maxPOS;};
            //Serial.println(SetPoint); //to be removed
            break;
        }
    }
    //no return
    break;

case 14: //***** RESET TIVA *****
    //Serial.println(F("Reset"));
    //delay(1);
    //digitalWrite(RSTpin, LOW); //reset function
    break;

case 15: //***** GET Coeff *****
    Serial.println(coeff);
    break;

case 16: //***** GET controller periode *****
    Serial.println(Timetable[1]-Timetable[0]);
    break;

case 17: //***** SET if derivative is require *****
    //Serial.println("Set SetPoint"); //to be removed
    while (Serial.available() == 0){}; //wait port free
    while (Serial.available() > 0) { //byte reception

```

```

out_rec = Serial.parseInt();
if (Serial.read() == '\n') {
    //0 = position output is require
    //1 = derivative output is require
    //2 = current output is require
    //Serial.println(SetPoint); //to be removed
    break;
}
}
//no return
break;

case 18: //***** SET Steptime or Starttime *****
//Serial.println("Set SetPoint"); //to be removed
while (Serial.available() == 0){}; //wait port free
while (Serial.available() > 0) { //byte reception
    StepInitTime = Serial.parseInt(); //in ms
    if (Serial.read() == '\n') {
        if (StepInitTime > 500) {StepInitTime = 500;}; //limits
        if (StepInitTime < 10) {StepInitTime = 10;};
        //Serial.println(StepInitTime); //to be removed
        break;
    }
}
//no return
break;

case 19: //***** SET Sampling Time for open loop *****
//Serial.println("Set SetPoint"); //to be removed
while (Serial.available() == 0){}; //wait port free
while (Serial.available() > 0) { //byte reception
    OpenLoopSampling = Serial.parseInt(); //in ms
    if (Serial.read() == '\n') {
        if (OpenLoopSampling > 10) {OpenLoopSampling = 10;}; //limits
        if (OpenLoopSampling < 1) {OpenLoopSampling = 1;};
        Serial.println(OpenLoopSampling); //to be removed
        break;
    }
}
//no return
break;

case 20: //***** GET MOTOR POSITION IN BITS *****
//Serial.println("Get Position"); //to be removed
POS = analogRead(SensorPos1);
//Serial.println(POS);
Serial.println(POS);
break;

case 21: //***** CLOSE LOOP STEP RESPONSE DISCRETE PID *****
PWMsetDC(50.0); // set Pwm dutycycle value to 50% = no torque
i = 1; //index init
j = 1;

```

```

Timetable[0] = 0;          //elapsed time index init (last time index)
Timetable[1] = 0;          //elapsed time index init (current time index)
//digitalWrite(DIR,LOW); //set direction to anticlockwise
INT = 0;                   //error integral initialization
L_ERR = 0;                 //last error initialization
x[0] = 0;                  //initial time
t_init = millis();         //init current time
y[0] = analogRead(SensorPos1); //initial position
z[0] = 0;                  //initial command
//Serial.println(POS); //to be removed
digitalWrite(ENABLE_M1,HIGH); //enable motor 1
//Serial.println(i);

delay(StepInitTime);       //wait before step start
while (i < 1500) {          //time limit for control loop in ms => 1,5s
    Timetable[1] = int(millis()-t_init); //get elapsed time
    POS = analogRead(SensorPos1);        //get current position
    CUR = analogRead(SensorCur);         //get motor current
    ERR = SetPoint - POS;                 //Error evaluation
    U = Ad*U + Bd*float(ERR) + Cd*float(L_ERR); //discrete command calculus
    COM = (-coeff*(U)*4.16)+50;           //Conversion for volts
    if (COM > 99.9){                       //PWM limits
        COM = 99.9;
    } else if (COM < 0.1){
        COM = 0.1;
    };
    PWMsetDC(COM); //update Pwm value
    if ((i-1) % 4 == 0){ //1 record data each 4
        x[j] = Timetable[1];
        y[j] = POS;
        z[j] = int(COM);
        j++;
    }
    TP1 = y[j-1] - y[j-3]; //ultimate output position difference
    TP2 = y[j-3] - y[j-6]; //last output position difference
    if ((j > MinSimTime) && (abs(TP1) < error_var) && (abs(TP2) < error_var)){
        //Serial.println("STOP"); //to be removed
        break; // STOP if steady state or a physical limit is reach
    }
    L_ERR = ERR; //error update
    Timetable[0] = Timetable[1]; //Elapsed time update
    //delayMicroseconds(500); //loop delay
    delay(Ted); //loop delay fixed by server
    i++; //Next step
}

digitalWrite(ENABLE_M1,LOW); //disable motor 1
//Serial.println(j);

print3DtableJSON(x,y,z,j); //send time, position and 10 x command
break;

case 22: //***** SET Ad VALUE *****
//Serial.println("Set Ad"); //to be removed
while (Serial.available() == 0){}; //wait port free
while (Serial.available() > 0) { //byte reception

```

```

    Ad = Serial.parseFloat(); //Ad received
    if (Serial.read() == '\n') {
        if (Ad > 100){Ad = 100;};
        if (Ad < 0){Ad = 0;};
        break;
    }
}
//no return
break;

case 23: //***** SET Bd VALUE *****
//Serial.println("Set Bd"); //to be removed
while (Serial.available() == 0){}; //wait port free
while (Serial.available() > 0) { //byte reception
    Bd = Serial.parseFloat(); //Bd received
    if (Serial.read() == '\n') {
        if (Bd > 100){Bd = 10;};
        if (Bd < 0){Bd = 0;};
        break;
    }
}
//no return
break;

case 24: //***** SET Cd VALUE *****
//Serial.println("Set Kp"); //to be removed
while (Serial.available() == 0){}; //wait port free
while (Serial.available() > 0) { //byte reception
    Cd = Serial.parseFloat(); //Cd received
    if (Serial.read() == '\n') {
        if (Cd > 10){Cd = 10;};
        if (Cd < 0){Cd = 0;};
        break;
    }
}
//no return
break;

case 25: //***** SET Ted VALUE *****
//Serial.println("Set Ted"); //to be removed
while (Serial.available() == 0){}; //wait port free
while (Serial.available() > 0) { //byte reception
    Ted = Serial.parseInt(); //Ted received
    if (Serial.read() == '\n') {
        if (Ted > 100){Ted = 100;};
        if (Ted < 0){Ted = 0;};
        break;
    }
}
//no return
break;

case 26: //***** SET anti-windup on *****

```

```
//Serial.println("Rotation anticlockwise"); //to be removed
AWU = true; //Anti-windup ON
//no return
break;

case 27: //***** SET anti-windup off *****
//Serial.println("Rotation anticlockwise"); //to be removed
AWU = false; //Anti-windup OFF
//no return
break;

} //end switch
} //end if
} //end while
} //end main loop
```

BORRADOR

Annexe 2 : SERVER CODE

```
var http = require('http').createServer(handler); //require http server, and create server with function handler()
var fs = require('fs'); //require filesystem module
var io = require('socket.io')(http); //require socket.io module and pass the http object (server)
var url = require('url');
var FormData = require('form-data');
var AuthSessionID = 123456789;
var PortOK = false;
const { SerialPort } = require('serialport');
var derreq = 0;
var ExtDAQ = true; //***** set arduino/TIVA DAQ or Rpi GPIO *****

var FormData = require('form-data');
var command = new FormData();
var XMLHttpRequest = require("xmlhttprequest").XMLHttpRequest;

const Gpio = require('pigpio').Gpio;
const LED = new Gpio(12, {mode: Gpio.OUTPUT});
const pushButton = new Gpio(17, {
  mode: Gpio.INPUT,
  pullUpDown: Gpio.PUD_DOWN,
  edge: Gpio.EITHER_EDGE
});
const mot_pwm = new Gpio(20, {mode: Gpio.OUTPUT});
const mot_en = new Gpio(16, {mode: Gpio.OUTPUT});
const mot_dir = new Gpio(21, {mode: Gpio.OUTPUT});
const ADS1115 = require('ads1115');
const i2c = require('i2c-bus');

var Kp, Ki, Kd = 0.0;
var PIDRef = 0.0;

const { promisify } = require('util')
const sleep = promisify(setTimeout)

var ref_Value = 50; //ref: 0-100%
var trans = 0; //transaction between arduino and server 0 = "OK", 1 = "POS"

//table for step response
var X = [];
var Y = [];

if (!ExtDAQ){
  mot_en.digitalWrite(1);
  mot_pwm.pwmWrite(dutyCycle);
};

var dutyCycle = Math.trunc((ref_Value/100)*255); //duty cycle: 0-255

//list of available ports and connection if Texas is available
const { ReadlineParser } = require('@serialport/parser-readline')
```

```

const port = new SerialPort({ path: '/dev/ttyACM0', baudRate: 115200}); // port at /dev/ttyACM1 o /dev/ttyACM0
port.setEncoding('utf8');

//var Readline = SerialPort.parsers.Readline;
//var parser = new Readline(); // make a new parser to read ASCII lines
//port.pipe(parser); // pipe the serial stream to the parser
//port.on('error', function(err) {
//    //console.log('Error: ', err.message)
//})

const parser = port.pipe(new ReadlineParser({ delimiter: '\r\n' }))
//parser.on('data', console.log)

http.listen(8080); //listen to port 8080
console.log("Server listening...");
//var lab_state;

function getJSON(arrayX,arrayY) {    //convert two arrays in JSON format for sending to client
    var JSON = "[";
    //should arrayX length equal arrayY lenght and both against null
    if (arrayX != null && arrayY != null && arrayX.length == arrayY.length) {
        for (var i = 0; i < arrayX.length; i++) {
            JSON += "{";
            JSON += "x:" + arrayX[i] + ",";
            JSON += "y:" + arrayY[i];
            JSON += "},";
        }
    }
    JSON += "]"
    JSON = Function("return " + JSON + " ");
    return JSON();
}

function handler (req, res) { //create server
    var q = url.parse(req.url, true).query; //example: http://localhost:8080/?id=123456789
    var client = q.id; //get client ID
    if (client == AuthSessionID) { //verify ID authorization
        console.log("Client allowed:",client);
        fs.readFile(__dirname + '/public/index2.html', function(err, data) { //read file index2.html in public
            folder
            if (err) {
                res.writeHead(404, {'Content-Type': 'text/html'}); //display 404 on error
                return res.end("404 Not Found");
            }

            res.writeHead(200, {'Content-Type': 'text/html'}); //write HTML
            res.write(data); //write data from index.html
            return res.end();
        });
    } else {
        console.log("Client not allowed:",client);
        res.writeHead(404, {'Content-Type': 'text/html'}); //display 404 on error
        return res.end("Client not allowed");
    }
}

```

```

io.sockets.on('connection', function (socket) { // WebSocket Connection

    var lightvalue = 0; //static variable for current LED status

    console.log("Page loaded");

    //pushButton.watch(function (err, value) { //Watch for hardware interrupts on pushButton exemple
    // if (err) { //if an error
    //     console.error('There was an error', err); //output error message to console
    //     return;
    // }
    // lightvalue = value;
    // socket.emit('light', lightvalue); //send button status to client
    //});

    port.write('12\n'); //get initial motor position
    trans =1; //wait motor position data to update webpage

    parser.on('data', function (data) { /*** Process data from Arduino and extract expected information ***
        if (trans == 0){ //Standard OK/ERROR response
            if (data == "OK\r") {
                console.log('OK'); //OK if there is no error
            } else {
                console.log('Trans ERROR'); //there is an error... doing something
                port.write('14\n'); //Arduino/TIVA Hard Reset
            }
        } else if (trans == 1){ //Position number reception
            console.log('Position: ',Number(data)); //echo position number
            socket.emit('anl_value', Number(data).toFixed(1)); //send position to client (in °)
        } else if (trans == 2){ //Position or velocity table reception
            //console.log('data: ',data);
            let datos = JSON.parse(data); //parse data into JSON format
            // send data to web server
            socket.emit('data', datos); //send data to local client
            //if (derreq==0){socket.emit('anl_value', (student[student.length-1].y).toFixed(1));} //send
ultimate value to client position display
        } else if (trans == 3){ //Coefficient numbers Ki,
Kp, Kd, etc. reception
            console.log('value: ',Number(data)); //echo position number... to be used
        }
    });

    socket.on('light', function(data1) { //get light switch status from client
        lightvalue = data1;
        if (lightvalue != LED.digitalRead()) { //only change LED if status has changed
            LED.digitalWrite(lightvalue); //turn LED on or off
        }
    });

    socket.on('outputreq', function(data1) { //get derivative switch status from client
        if (ExtDAQ){
            port.write('17\n'); //Send Set Output Request command to ExtDAQ/TIVA: 0, 1 or 2
            port.write(data1.toString()+"\n"); //Send "0" = no derivative requested, "1" = derivative
requested, "2" = current requested
            derreq = data1;
            //no return
        } else {
            //not implemented
        }
    });

```

```

    }

    console.log("Output: ",data1);
});

socket.on('selSampling', function(data1) { //get sampling value from client

    if (ExtDAQ){

        port.write('19\n'); //Send Set Sampling Time to ExtDAQ/TIVA

        port.write(data1.toString()+'\n'); //Send 1-10 ms

        //no return

    } else {

        //not implemented

    }

    console.log(data1);
});

socket.on('mot_en', function(data2) { //receive motor enable checkbox value from client

    mot_en_value = !data2; //toggle value for Rpi GPIO

    console.log('data from client:', data2); //echo state

    if (ExtDAQ){

        if (data2 == 0){

            port.write('6\n'); //Stop motor

            trans = 1; //wait current position

        } else {

            port.write('5\n'); //Start motor

            //no return trans = 0;

        }

    }

    } else {

        if (mot_en_value != mot_en.digitalRead()) { //only change LED if status has changed

            mot_en.digitalWrite(mot_en_value); //turn motor enable on or off

        }

    }

});

socket.on('mot_dir', function(data3) { //receive motor direction checkbox status from client

    mot_dir_value = data3;

    console.log('data from client:', mot_dir_value); //echo state

    if (ExtDAQ){

        if (mot_dir_value == 0){

            port.write('8\n'); //Send direction command

            //no return trans = 0;

        } else {

            port.write('7\n'); //Send direction command

            //no return trans = 0;

        }

    }

    } else {

        if (mot_dir_value != mot_dir.digitalRead()) { //only change LED if status has changed

            mot_dir.digitalWrite(mot_dir_value); //turn motor enable on or off

        }

    }

});

socket.on('AWU', function(data4) { //receive motor direction checkbox status from client

    AWU_value = data4;

    console.log('data from client:', AWU_value); //echo state

    if (ExtDAQ){

```

```

        if (AWU_value == 0){
            port.write('26\n');          //Send AWU on
            //no return trans = 0;

        } else {
            port.write('27\n');          //Send AWU off
            //no return trans = 0;

        };

    } else {
        if (mot_dir_value != mot_dir.digitalRead()) { //only change LED if status has changed
            mot_dir.digitalWrite(mot_dir_value); //turn motor enable on or off
        };

    };

});

socket.on('ref_input', function(ref) { //receive voltaje from client as duty cycle 0-255
    dutyCycle = ref; //update duty cycle
    console.log("PMW: ", dutyCycle);
    if (ExtDAQ){
        port.write('4\n'); //Send Set PWM command to arduino/TIVA
        port.write(dutyCycle.toString()+'\n'); //Send Set PWM command to arduino/TIVA
        // no return
    } else {
        mot_pwm.pwmWrite(dutyCycle);
    }
});

socket.on('Kp_input', function(ref) { //receive Kp from client: real between 0-100
    if (ExtDAQ){
        port.write('9\n'); //Send Set PWM command to arduino/TIVA
        port.write(ref.toString()+'\n'); //Send Set PWM command to arduino/TIVA
        //no return
    }
    Kp = ref; //set local variable
    console.log("Kp",Kp);
});

socket.on('Ki_input', function(ref) { //receive Ki from client: real between 0-100
    if (ExtDAQ){
        port.write('10\n'); //Send Set PWM command
        port.write(ref.toString()+'\n'); //Send Set PWM command
        //no return
    }
    Ki = ref; //set local variable
    console.log("Ki",ref);
});

socket.on('Kd_input', function(ref) { //receive Kd from client: real between 0-100
    if (ExtDAQ){
        port.write('11\n'); //Send Set PWM command
        port.write(ref.toString()+'\n'); //Send Set PWM command
        //no return
    }
    Kd = ref;
    console.log("Kd",Kd);
});

socket.on('PID_input', function(ref) { //receive position reference from client (real number)
    //numero between 0-185°

```

```

    if (ExtDAQ) {
        port.write('13\n'); //Send Set PWM command
        port.write(ref.toString()+'\n'); //Send Set PWM command
        //no return
    }

    PIDRef = ref;

    console.log("PID ref",PIDRef);
});

socket.on('reset_pos', function(data5) { // ** Move motor to initial position
    var value = 0;

    if (ExtDAQ) { // ** for rapid systems **
        port.write('0\n'); //Send open loop step response command
        console.log('Reset pos. ');
        trans = 1; //wait motor Position return
    } else { // ** for slow systems **
        mot_pwm.pwmWrite(100);
        i2c.openPromisified(1).then(async (bus) => {
            LED.digitalWrite(true);
            mot_dir.digitalWrite(true); //return to 0 (antihour)
            mot_en.digitalWrite(false); //start motor
            const ads1115 = await ADS1115(bus);
            ads1115.gain = 1;
            var value = await ads1115.measure('0+GND');
            //console.log(value);
            const start = Date.now();
            while (value > 1250){
                value = await ads1115.measure('0+GND');
                if ((Date.now()-start) > 2000){break;};
            }
            mot_en.digitalWrite(true); //stop motor
            LED.digitalWrite(false);
            mot_dir.digitalWrite(false);
            socket.emit('anl_value', value/100);
            mot_pwm.pwmWrite(dutyCycle);
        });
    }
});

socket.on('Ctrl_go', function(data4) { //get step request from client
    port.write('2\n'); //Send closed loop step response command
    console.log('Go control');
    trans = 2; //return table
});

socket.on('GetPos', function(data4) { //get position request from client: without argument
    console.log('Request position:',data4);
    port.write('12\n'); //get initial motor position
    trans = 1; //wait motor position data to update webpage
});

socket.on('get_input', function(data4) { //get step request from client
    // ** STEP RESPONSE FUNCTION **
    if (ExtDAQ) { // ** rapid system **

```

```

port.write('1\n'); //Send open loop step response command
trans = 2; //return table
} else {
i2c.openPromisified(1).then(async (bus) => {
    X = [];
    Y = [];
    const ads1115 = await ADS1115(bus);
    ads1115.gain = 1
    //STEP RESPONSE
    LED.digitalWrite(true);
    mot_dir.digitalWrite(false); //hour dir
    mot_en.digitalWrite(false); //start motor
    const start = Date.now();
    for (let i = 0; i<40; i++){
        X.push(Date.now() - start);
        Y.push(await ads1115.measure('0+GND')/100);
        await sleep(4*i); //less data points near the steady state
        if ((i > 3) && (Y[i] - Y[i-1] < 1) && (Y[i-1] - Y[i-2] < 1)){
            break; // if steady state or a physical limit is reach
        }
    }
    mot_en.digitalWrite(true); //stop motor
    LED.digitalWrite(false);
    let output = getJSON(X,Y);
    //console.log(output);
    let outdata = JSON.stringify(output); //parse data
    fs.writeFileSync('datatest.json', outdata); //write file
    await sleep(100); // 100 ms delay between write and read
    let rawdata = fs.readFileSync('datatest.json'); //read file
    let student = JSON.parse(rawdata); //parse data
    console.log(student[student.length-1].y);
    socket.emit('data', student); //send data to cliente
    socket.emit('anl_value', student[student.length-1].y); //send ultimate value
    });
};

//socket.emit('anl_value', val); //test send value
//let rawdata = fs.readFileSync('datatest.json'); //read file
//let student = JSON.parse(rawdata); //parse data
//console.log(student);
//socket.emit('data', student); //send data
});
});

process.on('SIGINT', function () { //on ctrl+c
    LED.digitalWrite(0); // Turn LED off
    //LED.unexport(); // Unexport LED GPIO to free resources
    //pushButton.unexport(); // Unexport Button GPIO to free resources
    process.exit(); //exit completely
});

```

Annexe 3 : HTML CODE

```
<!DOCTYPE html>
<html lang="en">
<head>

  <title>Control Motor</title>

  <meta charset="utf-8" />

  <meta name="viewport" content="width=device-width, initial-scale=1">

  <link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/css/bootstrap.min.css">

  <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js"></script>

  <script src="https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/js/bootstrap.min.js"></script>

  <script> //data load
    var mydataPoints1 = [];
    var mydataPoints2 = [];
    var mydatos;

    function addData2D(datos) { //for 2D table : time, position speed or current
      mydataPoints1 = [];
      for (var i = 0; i < datos.length-2; i++) {
        mydataPoints1.push({
          x: datos[i].x,
          y: datos[i].y
        });
      }
      document.getElementById("data").value = datos[datos.length-1].y; //ultimate value in the data is always
the position
      //chart.render();
    };

    function addData3D(datos) { //for 3D table : time, position and voltage
      mydataPoints1 = [];
      mydataPoints2 = [];
      for (var i = 0; i < datos.length-2; i++) {
        mydataPoints1.push({
          x: datos[i].x,
          y: datos[i].y
        });
        mydataPoints2.push({
          x: datos[i].x,
          y: datos[i].z
        });
      }
      document.getElementById("data").value = datos[datos.length-1].y; //ultimate value in the data is always
the position
      //chart.render();
    };

    function downloadData(exportObj, exportName){
      //var dataStr = "data:text/json;charset=utf-8," +
      encodeURIComponent(JSON.stringify(exportObj));

      //const items = exportObj.items;

      const replacer = (key, value) => value === null ? '' : value; // specify how you want to handle
null values here

      //console.log(exportObj[0]);

      const header = Object.keys(exportObj[0]);

      let csv = exportObj.map(row => header.map(fieldName => JSON.stringify(row[fieldName],
replacer)).join(','));

      console.log(csv);
    }
  }
</script>

```

```

        csv.unshift(header.join(','));

        csv = csv.join('\r\n');

        var downloadLink = document.createElement("a");

        var blob = new Blob(["\u00eff", csv]);

        var url = URL.createObjectURL(blob);

        downloadLink.href = url;

        downloadLink.download = "DataDump.csv"; //Name the file here

        document.body.appendChild(downloadLink);

        downloadLink.click();

        document.body.removeChild(downloadLink);

    };

</script>

</head>

<body>

<div class="container">

<h2>Control motor laboratory</h2>

<p>This lab allows you to test the system in open loop (moving the motor and plot a step response) and to
implemente PID feedback or state feedback control.</p>

<ul class="nav nav-pills">

<li class="active"><a data-toggle="pill" href="#home" id="mymenu1">Home</a></li>

<li><a data-toggle="pill" href="#menu1" id="mymenu2">Open loop</a></li>

<li><a data-toggle="pill" href="#menu2" id="mymenu3">PID feedback</a></li>

<li><a data-toggle="pill" href="#menu3" id="mymenu4">State feedback</a></li>

<li><a data-toggle="pill" href="#menu4" id="mymenu5">Discrete PID control</a></li>

</ul>

<div class="tab-content">

<div id="home" class="tab-pane fade in active">

<div class="panel panel-default">

<div class="panel-heading">Laboratory testing</div>

<div class="panel-body">

<div class="checkbox">

<p>Here, you can probe the interface and move the motor. The motor supply
is 12 V.</p>

<label class="checkbox-inline"><input type="checkbox" id="light">Plant
Light</label>

<label class="checkbox-inline"><input type="checkbox" id="mot_en">Motor
On/Off</label>

<label class="checkbox-inline"><input type="checkbox" id="mot_dir">Motor
Direction</label><br>

</div>

<div class="panel panel-default" style="width: 250px;">

<div class="panel-heading">Motor voltage (0-80%)</div>

<div class="slidecontainer">

<input type="range" min="0" max="80" value="20" step="5"

class="slider" id="voltRange">

</div>

</div>

</div>

</div>

<div id="menu1" class="tab-pane fade">

<div class="panel panel-default">

<div class="panel-heading">Open Loop Step response</div>

<div class="panel-body">

<div class="media">

```

```

<div class="media-left">
    
</div>
<div class="media-body">
    <h4 class="media-heading">Open loop system</h4>
    <p>Here, you can reset the initial motor position
(Return to Zero button) and, after, launch an Open Loop step response test. You can also see the current position.
    Notice that the input is a voltage and the output is a
position in ° or a speed in °/s.</p>
</div>
</div>
<p></p>
<input type="button" class="btn btn-info" id="reset_pos" value="Return to Zero"
style="width: 250px;">
<p></p>
<div class="input-group">
    <span class="input-group-addon" style="width: 150px;">Step amplitude
(V)</span>
    <input type="number" class="form-control" id="inputref" name="refvalue"
min="0.0" max="7.5" maxlength="3" value="5.0" placeholder="2.5" style="width: 100px;">
</div>
<input type="button" class="btn btn-info" id="step" value="Step response"
style="width: 250px;">
<p></p> Chart output:
<label class="radio-inline"><input type="radio" id="posoption" name="optradio"
checked>position</label>
<label class="radio-inline"><input type="radio" id="speedoption"
name="optradio">Speed</label>
<label class="radio-inline"><input type="radio" id="currentoption"
name="optradio">Current</label>
<p></p>
<div class="form-group">
    <label for="sel1">Base Sampling Time (ms):</label>
    <select class="form-control" id="sel1">
        <option value = 1>1</option>
        <option value = 2>2</option>
        <option value = 5 selected>5</option>
        <option value = 10>10</option>
    </select>
</div>
</div>
</div>
<div id="menu2" class="tab-pane fade">
    <div class="panel panel-default">
        <div class="panel-heading">Closed Loop Step response</div>
        <div class="panel-body">
            <div class="media">
                <div class="media-left">
                    
                </div>
                <div class="media-body">
                    <h4 class="media-heading">Parallel PID</h4>
                    <p>This is the parallel PID structure used in this
interface. With this structure you can cancel one or more effects by putting a 0 in the correspondng box.</p>
                </div>
            </div>
        </div>
    </div>

```

```

        </div>

        <p>Here, you can test a parallel PID control</p>

        <div class="input-group">

            <span class="input-group-addon" style="width: 150px;" data-
toggle="tooltip" title="Kp">Proportional gain</span>

            <input type="number" class="form-control" id="Kp" name="Kpvalue"
min="0" max="100" maxlength="3" value="0.0" placeholder="1" style="width: 100px;">

        </div>

        <div class="input-group">

            <span class="input-group-addon" style="width: 150px;" data-
toggle="tooltip" title="Ki">Integral gain    </span>

            <input type="number" class="form-control" id="Ki" name="Kivalue"
min="0" max="100" maxlength="3" value="0.0" placeholder="0" style="width: 100px;">

        </div>

        <div class="input-group">

            <span class="input-group-addon" style="width: 150px;" data-
toggle="tooltip" title="Kd">Derivative gain</span>

            <input type="number" class="form-control" id="Kd" name="Kivalue"
min="0" max="100" maxlength="3" value="0.0" placeholder="0" style="width: 100px;">

        </div>

        <div class="input-group">

            <span class="input-group-addon" style="width: 150px;" data-
toggle="tooltip" title="5-175">Input reference (°)</span>

            <input type="number" class="form-control" id="PIDRefvalue"
name="Refvalue" min="5" max="175" maxlength="3" value="50" placeholder="0" style="width: 100px;">

        </div>

        <div class="input-group">

            <input type="button" class="btn btn-info" id="reset_pos2"
value="Return to Zero" style="width: 250px;"><br>

            <input type="button" class="btn btn-info" id="CtrlTest"
value="Control test" style="width: 250px;">

        </div>

        <div class="checkbox">

            <label class="checkbox-inline"><input type="checkbox"
id="AWU">Anti-windup</label>

        </div>

    </div>

    <div>

        <div id="menu3" class="tab-pane fade">

            <div class="panel panel-default">

                <div class="panel-heading">State Space control</div>

                <div class="panel-body">

                    <p>Here, you can test a State Space control with the state vector  $X = [X_1; X_2; X_3]$  where  $\dot{X} =$ 
 $w/s$ ,  $X_2 = i$ ,  $X_3 = w$  ( $w$  is the angular speed).

                    The command is :  $u = - K.X$  with  $K = [K_1 \ K_2 \ K_3]$ .</p>

                    <div class="input-group">

                        <span class="input-group-addon" style="width: 150px;">K1
gain</span>

                        <input type="number" class="form-control" id="Kp" name="Kpvalue"
min="0" max="100" maxlength="3" value="0.0" placeholder="1" style="width: 100px;">

                    </div>

                    <div class="input-group">

                        <span class="input-group-addon" style="width: 150px;">K2 gain

                        <input type="number" class="form-control" id="Ki" name="Kivalue"
min="0" max="100" maxlength="3" value="0.0" placeholder="0" style="width: 100px;">

                    </div>

                    <div class="input-group">

                        <span class="input-group-addon" style="width: 150px;">K3
gain</span>

```

```

        <input type="number" class="form-control" id="Kd" name="Kvalue"
min="0" max="100" maxlength="3" value="0.0" placeholder="0" style="width: 100px;">
    </div>
    <div class="input-group">
        <span class="input-group-addon" style="width: 150px;">Input
reference (^)</span>
        <input type="number" class="form-control" id="PIDRefvalue"
name="Refvalue2" min="0" max="180" maxlength="3" value="50" placeholder="0" style="width: 100px;">
    </div>
    <div class="input-group">
        <input type="button" class="btn btn-info" id="reset_pos2"
value="Return to Zero" style="width: 250px;"><br>
        <input type="button" class="btn btn-info" id="CtrlTest"
value="Control test" style="width: 250px;">
    </div>
</div>
</div>
<div id="menu4" class="tab-pane fade">
    <div class="panel panel-default">
        <div class="panel-heading">Discrete PID control</div>
        <div class="panel-body">
            <p>Here, you can test a discret control law  $u(k) = A*u(k-1) + B*e(k) + C*e(k-1)$  where "u" is
the controller output, "e" is the error input, setting  $u(k-1)$ ,  $e(k)$  and  $e(k-1)$  coefficients A, B, and C
respectively, as well as the period of the closed loop  $T_e$ .</p>
            <div class="input-group">
                <span class="input-group-addon" style="width: 150px;">A
gain</span>
                <input type="number" class="form-control" id="Ad" name="Avalue"
min="0" max="100" maxlength="3" value="0.0" placeholder="1" style="width: 100px;">
            </div>
            <div class="input-group">
                <span class="input-group-addon" style="width: 150px;">B gain
</span>
                <input type="number" class="form-control" id="Bd" name="Bvalue"
min="0" max="100" maxlength="3" value="0.0" placeholder="0" style="width: 100px;">
            </div>
            <div class="input-group">
                <span class="input-group-addon" style="width: 150px;">C
gain</span>
                <input type="number" class="form-control" id="Cd" name="Cvalue"
min="0" max="100" maxlength="3" value="0.0" placeholder="0" style="width: 100px;">
            </div>
            <div class="input-group">
                <span class="input-group-addon" style="width: 150px;"> $T_e$  period
(ms)</span>
                <input type="number" class="form-control" id="Te" name="Tevalue"
min="1" max="20" maxlength="3" value="1.0" placeholder="0" style="width: 100px;">
            </div>
            <div class="input-group">
                <span class="input-group-addon" style="width: 150px;">Input
reference (^)</span>
                <input type="number" class="form-control" id="PIDRefvalue"
name="Refvalue2" min="0" max="180" maxlength="3" value="50" placeholder="0" style="width: 100px;">
            </div>
            <div class="input-group">
                <input type="button" class="btn btn-info" id="reset_pos2"
value="Return to Zero" style="width: 250px;"><br>
                <input type="button" class="btn btn-info" id="CtrlTest"
value="Control test" style="width: 250px;">
            </div>
        </div>
    </div>
</div>

```

```

        </div>
    </div>
</div>

<div class="panel-group">
    <div class="panel panel-default">
        <div class="panel-heading">Plot and data panel</div>
        <div class="panel-body">
            <p>You can get particular values by passing mouse over the curve.
                <div class="input-group">
                    <span class="input-group-addon" style="width: 150px;">Current
position (°)</span>
                    <input type="number" class="form-control" id="data" name="lname"
maxlength="3" style="width: 100px;" disabled>
                </div>
                <div id="chartContainer" style="height: 370px; width: 100%;"></div>
                <input type="button" class="btn btn-info" id="downloadAnchorElem"
onclick="downloadData(mydatos, 'datafile')" value="Download data">
            </div>
        </div>
    </div>
</div>


</div>

<script src="https://canvasjs.com/assets/script/jquery-1.11.1.min.js"></script>
<script src="https://canvasjs.com/assets/script/canvasjs.min.js"></script>

<script src="https://cdnjs.cloudflare.com/ajax/libs/socket.io/2.0.3/socket.io.js"></script> <!-- include socket.io
client side script -->

<script>
var socket = io(); //load socket.io-client and connect to the host that serves the page
document.getElementById("downloadAnchorElem").disabled = true;
var changePos = document.getElementById("data");
var Refchart = 0;
var Chart_title = "";
var SetpointText = "";
var StepTime = 50; //step time in ms
var Maxvoltage = 12; //max voltage = 12V or 5V is the voltage of the motor supply
var Ytitle = "Position";
var Ysuffix = "°";
var Menu = 1;

window.addEventListener("load", function(){ //when page loads
    var lightbox = document.getElementById("light");
    var motbox = document.getElementById("mot_en");
    var dirbox = document.getElementById("mot_dir");
    var stepbtn = document.getElementById("step");
    var Ctrlbtn = document.getElementById("CtrlTest");
    var resetbtn = document.getElementById("reset_pos");
    var resetbtn2 = document.getElementById("reset_pos2");
    var volt_ref = document.getElementById("inputref");
    var Kp_ref = document.getElementById("Kp");
    var Ki_ref = document.getElementById("Ki");
    var Kd_ref = document.getElementById("Kd");
    var PID_ref = document.getElementById("PIDRefvalue");
    var slider = document.getElementById("voltRange");
    var positionOn = document.getElementById("posoption"); //radio button

```

```

var speedOn = document.getElementById("speedoption"); //radio button
var currentOn = document.getElementById("currentoption"); //radio button
var selSampling = document.getElementById("sel1");
var Menu1 = document.getElementById("mymenu1");
var Menu2 = document.getElementById("mymenu2");
var Menu3 = document.getElementById("mymenu3");
var Menu4 = document.getElementById("mymenu4");
var Menu5 = document.getElementById("mymenu5");
var AWUonoff = document.getElementById("AWU");
var value = 50;
var PWMvalue = Math.round(slider.value*2.55); //PWM dutycycle 0-255
var Refvalue = PID_ref.value; //position 0-185°
volt_ref.max = Maxvoltage;

Menu1.addEventListener("click", function() { //add event listener for when Menu Manual click
    console.log(1); //send button status to server (as 1 or 0)
    Menu = 1;
});

Menu2.addEventListener("click", function() { //add event listener for when Menu Open Loop click
    console.log(2); //send button status to server (as 1 or 0)
    Menu = 2;
});

Menu3.addEventListener("click", function() { //add event listener for when Menu Closed Loop click
    console.log(3); //send button status to server (as 1 or 0)
    Menu = 3;
    socket.emit("outputreq", 0); //Request position plot
});

Menu4.addEventListener("click", function() { //add event listener for when Menu Space State click
    console.log(4); //send button status to server (as 1 or 0)
    Menu = 4;
    socket.emit("outputreq", 0); //Request position plot
});

Menu5.addEventListener("click", function() { //add event listener for when Menu Discrete click
    console.log(5); //send button status to server (as 1 or 0)
    Menu = 5;
    socket.emit("outputreq", 0); //Request position plot
});

lightbox.addEventListener("click", function() { //add event listener for when checkbox click
    socket.emit("light", Number(this.checked)); //send button status to server (as 1 or 0)
});

positionOn.addEventListener("click", function() { //add event listener for when radiobutton Position click
    socket.emit("outputreq", 0); //send button status to server (as 1 or 0)
});

speedOn.addEventListener("click", function() { //add event listener for when radiobutton Speed click
    socket.emit("outputreq", 1); //send button status to server (as 1 or 0)
});

currentOn.addEventListener("click", function() { //add event listener for when radiobutton Current click
    socket.emit("outputreq", 2); //send button status to server (as 1 or 0)
});

AWUonoff.addEventListener("click", function() { //add event listener for when checkbox AWD click
    socket.emit("AWU", Number(this.checked)); //send button status to server (as 1 or 0)
});

selSampling.addEventListener("change", function() { //add event listener for when Sampling checkbox click

```

```

        socket.emit("selSampling", Number(this.value)); //send button status to server (as 1 or 0)
    });
    changePos.addEventListener("change", function() { //add event listener for when checkbox click
        socket.emit("Get_Pos", 1); //request position
    });
    motbox.addEventListener("click", function() { //add event listener for when checkbox click
        socket.emit("ref_input", PWMvalue); //update current PWM dt value
        socket.emit("mot_en", Number(this.checked)); //send button status to server (as 1 or 0)
    });
    dirbox.addEventListener("click", function() { //add event listener for when checkbox click
        socket.emit("mot_dir", Number(this.checked)); //send button status to server (as 1 or 0) ON/OFF
    });
    stepbtn.addEventListener("click", function() { //add event listener for OL Step Response button click
        socket.emit("ref_input", PWMvalue); //update current PWM dt value
        if (speedOn.checked == true){
            socket.emit("outputreq", 1);
            Ytitle = "Speed";
            Ysuffix = "°/s";
        }
        if (currentOn.checked == true){
            socket.emit("outputreq", 2);
            Ytitle = "Current";
            Ysuffix = "mA";
        }
        if (positionOn.checked == true){
            socket.emit("outputreq", 0);
            Ytitle = "Position";
            Ysuffix = "°";
        }
        Refchart = 0;
        SetpointText = "";
        Chart_title = "Open loop step response (u=" + (PWMvalue*(Maxvoltage/255)).toFixed(1) + " V)";
        socket.emit("get_input", Number(this.checked)); //send button status to server (as 1 or 0)
    });
    Ctrlbtn.addEventListener("click", function() { //add event listener for CL Control button click
        socket.emit("PID_input", Refvalue);
        Refchart = Refvalue;
        SetpointText = "Setpoint";
        Chart_title = "PID closed loop response";
        socket.emit("Ctrl_go", Number(this.checked)); //send button status to server (as 1 or 0)
    });
    resetbtn.addEventListener("click", function() { //add event listener for open loop reset button click
        socket.emit("reset_pos", Number(this.checked)); //send button status to server (as 1 or 0)
    });
    resetbtn2.addEventListener("click", function() { //add event listener for reset PID button click
        socket.emit("reset_pos", Number(this.checked)); //send button status to server (as 1 or 0)
    });
    volt_ref.addEventListener("change", function() { //add event listener for volt reference change
        if (this.value < 0) this.value = 0; //limits 0 - Max voltage
        if (this.value > Maxvoltage) this.value = Maxvoltage;
        PWMvalue = Math.round(this.value*255/Maxvoltage); //convert volts to 0-255 value
        slider.value = Math.round(this.value*8.33)
    });

```

```

Kp_ref.addEventListener("change", function() { //add event listener for when value changes
    if (this.value < 0) this.value = 0; //limits 0 - 100
    if (this.value > 100) this.value = 100;
    socket.emit("Kp_input", this.value); //send 10*Kp value to server
});
Ki_ref.addEventListener("change", function() { //add event listener for when value changes
    if (this.value < 0) this.value = 0; //limits 0 - 100
    if (this.value > 100) this.value = 100;
    socket.emit("Ki_input", this.value); //send 10*Ki value to server
});
Kd_ref.addEventListener("change", function() { //add event listener for when value changes
    if (this.value < 0) this.value = 0; //limits 0 - 100
    if (this.value > 100) this.value = 100;
    socket.emit("Kd_input", this.value); //send 10*Kd value to server
});
PID_ref.addEventListener("change", function() { //add event listener for when value changes
    if (this.value < 0) this.value = 0.0; //limits 0 - 185°
    if (this.value > 185) this.value = 185.0;
    Refvalue = this.value;
    Ref_chart = Refvalue;
});
slider.addEventListener("change", function() { //when slider change between 0-100
    PWMvalue = Math.round(this.value*2.55);
    socket.emit("ref_input", PWMvalue); //Send PWM value to server
    volt_ref.value = Math.round(this.value*Maxvoltage/255); //convertt to volts
});
});

socket.on('anl_value', function (data) { //receive position from server in ° to switch buttons
    document.getElementById("data").value = data; //change text according to data from Raspberry Pi
    console.log(data);
});

socket.on('data', function (datos) { //get data from server
    //console.log(datos);
    mydatos = datos;
    //addData(datos); //get dataPoints
    if (Menu==2){ //***** open loop test *****
        addData2D(datos); //get 2D table dataPoints
        var chart = new CanvasJS.Chart("chartContainer", {
            theme: "light2",
            zoomEnabled: true,
            zoomType: "xy",
            animationEnabled: true,
            exportEnabled: true,
            title: {
                text: Chart_title
            },
            axisX: {
                title: "Time",
                titleFontColor: "#4F81BC",
                minimum: 0,
                suffix : " ms",
            }
        });
    }
}

```

```

        gridColor: "lightblue" ,
gridThickness: 2,
        gridDashType: "dot",
        crosshair: {
            enabled: true,
            snapToDataPoint: true
        },
        stripLines: [{
            value: StepTime,
            label: "Start",
            labelAlign: "near",
            lineDashType: "dash"
        }]
    },
    axisY: {
        title: Ytitle,
        titleFontColor: "#4F81BC",
        lineThickness:1,
        suffix : Ysuffix,
        gridThickness: 2,
        gridDashType: "dot",
        crosshair: {
            enabled: true
        },
    },
    data: [{
        type: "line",
        name: "Output",
        showInLegend: true,
        markerSize: 0,
        xValueFormatString: "#### ms",
        yValueFormatString: "#,##0.00"+Ysuffix,
        dataPoints: mydataPoints1
    }]
});
};
if (Menu==3){ //***** closed loop test *****
addData3D(datos); //get 3D table dataPoints
var chart = new CanvasJS.Chart("chartContainer", {
    theme: "light2",
    zoomEnabled: true,
    zoomType: "xy",
    animationEnabled: true,
    exportEnabled: true,
    title: {
        text: Chart_title + " (click in legend to hide curve)"
    },
    legend: {
        cursor: "pointer",
        itemclick: function (e) {
            if (typeof (e.dataSeries.visible) === "undefined" || e.dataSeries.visible) {
                e.dataSeries.visible = false;
            } else {

```

```

        e.dataSeries.visible = true;
    }

    e.chart.render();
}

},

axisX: {

    title: "Time",

    titleFontColor: "#4F81BC",

    minimum: 0,

    suffix : " ms",

    gridColor: "lightblue" ,

gridThickness: 2,

    gridDashType: "dot",

    crosshair: {

        enabled: true,

        snapToDataPoint: true

    },

},

axisY: {

    title: Ytitle,

    lineColor: "#4F81BC",

    tickColor: "#4F81BC",

    labelFontColor: "#4F81BC",

    titleFontColor: "#4F81BC",

    lineThickness:1,

    suffix : Ysuffix,

    gridThickness: 2,

    gridDashType: "dot",

    crosshair: {

        enabled: true

    },

    stripLines: [{

        value: Refchart,

        label: SetpointText,

        labelAlign: "near",

        lineDashType: "dash"

    }]

},

axisY2: {

    title: "Command",

    lineColor: "#C24642",

    tickColor: "#C24642",

    labelFontColor: "#C24642",

    titleFontColor: "#C24642",

    lineThickness:1,

    suffix : "V",

    gridThickness: 2,

    gridDashType: "dot",

    crosshair: {

        enabled: true

    }

},

data: [{

```

```

        type: "line",
        name: "Output",
        color: "#4F81BC",
        showInLegend: true,
        markerSize: 0,
        xValueFormatString: "#### ms",
        yValueFormatString: "#,##0.00"+Ysuffix,
        dataPoints: mydataPoints1
    },
    {
        type: "stepLine",
        name: "Reference",
        showInLegend: true,
        markerSize: 0,
        xValueFormatString: "#### ms",
        yValueFormatString: "#,##0.00"+Ysuffix,
        dataPoints: [
            { x: mydataPoints1[0].x, y: mydataPoints1[0].y },
            { x: StepTime, y: Number(Refchart) },
            { x: mydataPoints1[mydataPoints1.length-1].x, y: Number(Refchart) }
        ]
    },
    {
        type: "line",
        name: "Command",
        color: "#C24642",
        axisYType: "secondary",
        showInLegend: true,
        markerSize: 0,
        xValueFormatString: "#### ms",
        yValueFormatString: "#,##0.00 V",
        dataPoints: mydataPoints2
    }
}

});
};
document.getElementById("downloadAnchorElem").disabled = false;
chart.render(); //chart
//socket.emit("Get_Pos", 1); //request position
});

</script>

<div id="chartContainer" style="height: 400px; width: 80%;"></div>
<script src="https://canvasjs.com/assets/script/jquery-1.11.1.min.js"></script>
<script src="https://canvasjs.com/assets/script/canvasjs.min.js"></script>

</body>
</html>

```